

# Factory

## Synthèse : Le Patron Factory Method

Le **Factory Method** est un patron de conception qui délègue la création d'objets à une méthode spécifique, plutôt que de les instancier directement avec `new`. Ce patron s'avère particulièrement utile lorsque la classe concrète de l'objet à créer n'est pas connue à l'avance, notamment dans une hiérarchie de classes.

## Problématique

Dans le développement d'une suite logicielle de gestion de documents, nous définissons les éléments suivants :

- **Document** : une classe abstraite servant de base pour tous les documents.
- **Application** : une classe abstraite de base pour les logiciels de la suite.
- Deux éditeurs spécifiques : **PDFEditor** (gestion des PDF) et **SimpleTextEditor** (gestion des textes ASCII).

L'objectif est d'implémenter la création de documents dans la classe `Application`, comprenant :

- La vérification des droits de l'utilisateur.
- La journalisation des actions effectuées.
- La création de documents selon leur type.

## Problème du Code Initial

Dans la première version, la méthode `create()` de `Application` utilise une logique conditionnelle ( `if-else` ) pour créer les documents. Cela entraîne plusieurs difficultés :

1. **Manque de maintenabilité** : l'ajout d'un nouveau type de document (Excel, PPT, etc.) nécessite de modifier `Application`.
2. **Dépendances fortes** : `Application` doit connaître tous les types de documents existants et futurs.
3. **Risque d'erreurs** : la complexité croissante des conditions rend le code difficile à comprendre et à déboguer.

## Solution : Factory Method

Pour résoudre ces problèmes, le patron **Factory Method** introduit une méthode abstraite `createDocument()` dans `Application`. Chaque sous-classe concrète (comme `PDFEditor` ou `SimpleTextEditor`) implémente cette méthode pour créer son type de document spécifique.

## Code Refactorisé

La classe `Application` devient :

```
abstract class Application {
    public Document create(String typeDocument) {
        if (!verifierDroits()) {
            throw new RuntimeException("Droits insuffisants");
        }
        logCreationDocument();
        return createDocument(typeDocument);
    }

    protected abstract Document createDocument(String typeDocument);

    private boolean verifierDroits() { return true; }
    private void logCreationDocument() {
        System.out.println("Création d'un nouveau document en cours...");
    }
}
```

Chaque sous-classe implémente `createDocument()` :

```
class PDFEditor extends Application {
    @Override
    protected Document createDocument(String typeDocument) {
        if ("PDF".equalsIgnoreCase(typeDocument)) {
            return new PDFDocument();
        } else {
            throw new RuntimeException("Type non pris en charge");
        }
    }
}
```

```

}

class SimpleTextEditor extends Application {
    @Override
    protected Document createDocument(String typeDocument) {
        if ("SimpleText".equalsIgnoreCase(typeDocument)) {
            return new SimpleTextDocument();
        } else {
            throw new RuntimeException("Type non pris en charge");
        }
    }
}

```

## Bénéfices

- **Extensibilité** : l'ajout d'un nouveau type de document nécessite uniquement une nouvelle sous-classe de `Application`, sans modifier le code existant.
- **Maintenabilité** : `Application` reste générique et indépendante des types de documents spécifiques.
- **Encapsulation** : la logique de création est isolée dans chaque sous-classe concernée.

## Conclusion

Le **Factory Method** réduit le couplage et facilite l'extension du code. Il applique le principe de **délégation** en confiant la création des objets aux sous-classes spécialisées. Ce patron excelle dans les architectures orientées objet où les classes concrètes évoluent fréquemment.

Le diagramme UML du **Factory Method** illustre cette organisation en montrant la relation entre la classe `Application`, ses sous-classes concrètes et les documents qu'elles créent.